NAME : LOGESHWARI S **REG NO : 192565040**

COURSE CODE :CSA5112 **SLOT : B**

EXPERIMENT 1

IMPLEMENTATION OF HASH FUNCTION – MD5

AIM

To implement the MD5 (Message-Digest Algorithm 5) cryptographic hash function using Python and generate the fixed-length message digest for a given input.

ALGORITHM

Step 1: Start the program.

Step 2: Import the hashlib module.

Step 3: Read the message from the user.

Step 4: Convert the message into bytes using UTF-8 encoding.

Step 5: Apply the MD5 hash function to the message.

Step 6: Generate the hexadecimal message digest.

Step 7: Display the original message and its MD5 hash value.

Step 8: Stop the program.

PROGRAM (Python)

```
import hashlib

message = input("Enter message: ")
digest = hashlib.md5(message.encode()).hexdigest()

print("Message:", message)
```

```
print("MD5 Hash:", digest)
```

INPUT

Enter message: Hello World

OUTPUT

Message: Hello World

MD5 Hash: b10a8db164e0754105b7a99be72e3fe5

RESULT

Thus, the MD5 hash function was implemented successfully, and the message digest was generated correctly for the given input message.

EXPERIMENT 2

IMPLEMENTATION OF HASH FUNCTION – SHA-512

AIM

To implement the SHA-512 (Secure Hash Algorithm 512-bit) cryptographic hash function using Python and generate a 512-bit message digest for a given input.

ALGORITHM

Step 1: Start the program.

Step 2: Import the hashlib module.

Step 3: Read the message from the user.

Step 4: Convert the message into bytes using UTF-8 encoding.

Step 5: Apply the SHA-512 hash function to the message.

Step 6: Generate the hexadecimal message digest.

Step 7: Display the original message and its SHA-512 hash value.

Step 8: Stop the program.

PROGRAM (Python)

```
import hashlib

message = input("Enter message: ")
digest = hashlib.sha512(message.encode()).hexdigest()

print("Message:", message)
print("SHA-512 Hash:", digest)
```

INPUT

Enter message: Hello World

OUTPUT

Message: Hello World

A SHA-512 hexadecimal digest is generated and displayed. Its length is 128 hexadecimal characters (512 bits).

RESULT

Thus, the SHA-512 hash function was implemented successfully, and the 512-bit message digest was generated correctly for the given input message.

EXPERIMENT 3

IMPLEMENTATION OF DIGITAL SIGNATURE

AIM

To implement a digital signature using a public-key cryptographic technique in Python and verify the authenticity and integrity of a message.

ALGORITHM

Step 1: Start the program.

Step 2: Generate an RSA public key and private key pair.

Step 3: Read the message from the user.

Step 4: Create a cryptographic hash of the message.

Step 5: Sign the message digest using the private key.

Step 6: Display the generated digital signature.

Step 7: Verify the signature using the public key and the original message.

Step 8: Display whether the signature is valid or invalid.

Step 9: Stop the program.

PROGRAM (Python)

```
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes

private_key = rsa.generate_private_key(public_exponent=65537, key_size=2048)
public_key = private_key.public_key()

message = input("Enter message: ").encode()

signature = private_key.sign(
```

```
    message,
    padding.PSS(mgf=padding.MGF1(hashes.SHA256()),
            salt_length=padding.PSS.MAX_LENGTH),
    hashes.SHA256()
)

print("Digital Signature generated successfully.")

try:
    public_key.verify(
        signature, message,
        padding.PSS(mgf=padding.MGF1(hashes.SHA256()),
                salt_length=padding.PSS.MAX_LENGTH),
        hashes.SHA256()
    )
    print("Signature Verification: VALID")
except Exception:
    print("Signature Verification: INVALID")
```

INPUT

Enter message: Hello World

OUTPUT

Digital Signature generated successfully.

Signature Verification: VALID

RESULT

Thus, the digital signature algorithm was implemented successfully, and the generated signature was verified as valid using the corresponding public key.